

Signal Processing, Machine Learning and Control

HW2

Sravanth Chowdary Potluri
und5uv
CS6762

September 23, 2025

1 Question 1: Properties of a Linear System

A system must satisfy two specific properties to be considered linear: **homogeneity** and **additivity**. A third property, **shift invariance**, is also typical of systems studied in signal processing but is not a requirement for linearity itself. When a system possesses all three properties, it is known as a Linear Time-Invariant (LTI) system.

Required Properties

- **Homogeneity (Scaling):** This property states that if an input signal $x[n]$ produces an output signal $y[n]$, then scaling the input signal by a constant factor k will result in the output signal also being scaled by the same factor. Mathematically, if $x[n] \rightarrow y[n]$, then $k \cdot x[n] \rightarrow k \cdot y[n]$.
- **Additivity:** This property dictates that if two separate input signals, $x_1[n]$ and $x_2[n]$, produce respective output signals $y_1[n]$ and $y_2[n]$, then applying the sum of the two input signals, $x_1[n] + x_2[n]$, will produce an output that is the sum of the individual outputs, $y_1[n] + y_2[n]$.

Typical Property

- **Shift Invariance:** While not a requirement for linearity, this property is very common. It means that the system's behavior does not change over time. If an input signal $x[n]$ results in an output $y[n]$, then a time-shifted version of the input, $x[n + s]$, will produce an identically time-shifted output, $y[n + s]$.

2 Question 2: Impulse Decomposition of Temperature Signal

The discrete-time temperature record $x[n] = (0, 0), (1, 80), (2, 85), (3, 90), (4, 87), (5, 88)$ can be decomposed as a weighted sum of shifted unit impulses. Figure 1 shows the original sequence, while Figure 2 presents each component $x_k[n] = x[k]\delta[n - k]$ that contributes to the reconstruction.

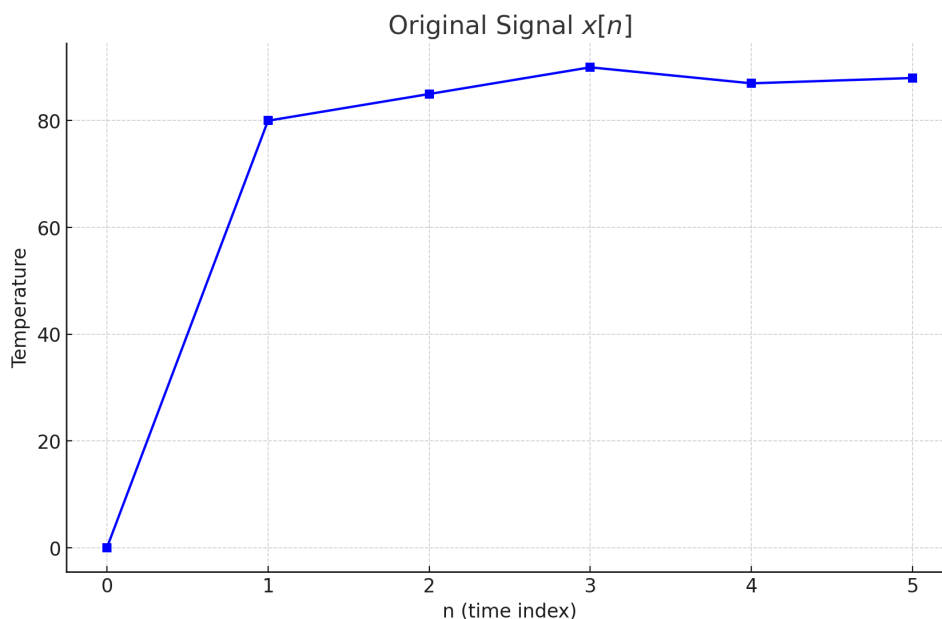


Figure 1: Original temperature sequence $x[n]$ measured at integer time indices.

3 Question 3: Convolution of $x[n]$ and $h[n]$

We are given the finite-length sequences (time/value pairs)

$$x[0..8] = \{0, -1, -1.5, 2, 1.5, 1.5, 0.8, 0, -0.5\}, \quad h[0..3] = \{1, -0.8, -0.5, -0.2\}.$$

The output is the discrete-time linear convolution

$$y[n] = (x * h)[n] = \sum_{k=-\infty}^{\infty} x[k] h[n-k], \quad 0 \leq n \leq 11.$$

Computing term-by-term (only nonzero products shown):

$$\begin{aligned} y[0] &= 0 \\ y[1] &= (-1)(1) = -1.00 \\ y[2] &= (-1)(-0.8) + (-1.5)(1) = -0.70 \\ y[3] &= (-1)(-0.5) + (-1.5)(-0.8) + (2)(1) = 3.70 \\ y[4] &= (-1)(-0.2) + (-1.5)(-0.5) + (2)(-0.8) + (1.5)(1) = 0.85 \\ y[5] &= (-1.5)(-0.2) + (2)(-0.5) + (1.5)(-0.8) + (1.5)(1) = -0.40 \\ y[6] &= (2)(-0.2) + (1.5)(-0.5) + (1.5)(-0.8) + (0.8)(1) = -1.55 \\ y[7] &= (1.5)(-0.2) + (1.5)(-0.5) + (0.8)(-0.8) = -1.69 \\ y[8] &= (1.5)(-0.2) + (0.8)(-0.5) + (-0.5)(1) = -1.20 \\ y[9] &= (0.8)(-0.2) + (-0.5)(-0.8) = 0.24 \\ y[10] &= (-0.5)(-0.5) = 0.25 \\ y[11] &= (-0.5)(-0.2) = 0.10 \end{aligned}$$

Thus, the result is

$$y[0..11] = \{0, -1, -0.7, 3.7, 0.85, -0.40, -1.55, -1.69, -1.20, 0.24, 0.25, 0.10\}.$$

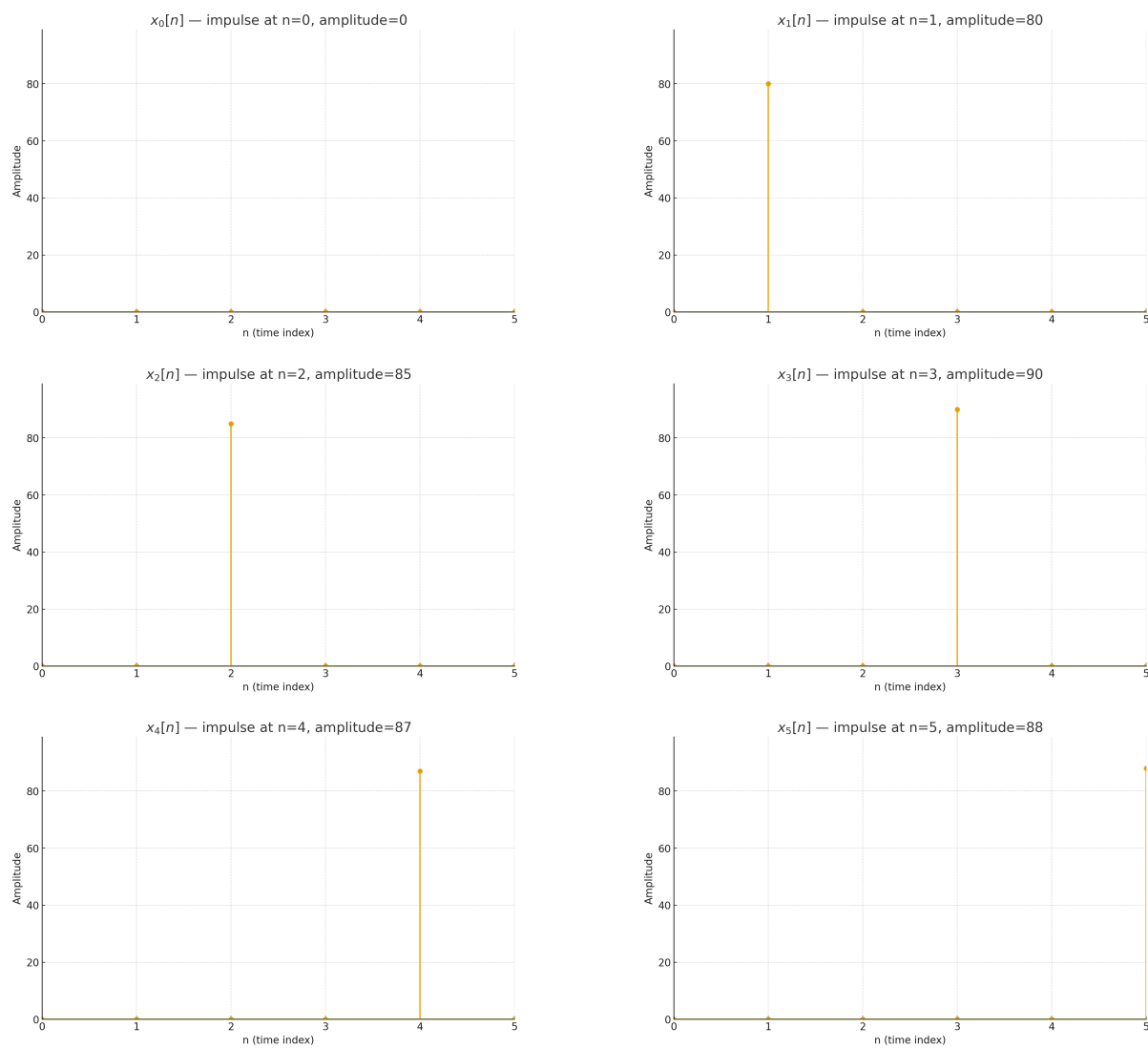


Figure 2: Impulse components $x_k[n]$ for $k = 0, \dots, 5$. Summing the six weighted impulses recreates the original signal.

4 Question 4 : DFT Program Trace for $K = 1$

The given program in BASIC has been converted to python to calculate the DFT and the trace for the given signal at $K = 1$ has been generated.

Python code used to generate the trace

```
import numpy as np

# Input signal XX at n = 0..5
XX = np.array([1, 1, 2, 1, 3, 2], dtype=float)
N = len(XX)

# Accumulators: real and imaginary parts of X[K]
REX = np.zeros(N)
IMX = np.zeros(N)

# Lines 310..380 in BASIC: two nested loops over K and I
for K in range(N):
    # 310
    for I in range(N):
        # 320
        ang = 2*np.pi*K*I/N
        c = np.cos(ang)
        s = np.sin(ang)

        # ---- TRACE only for K=1 (before and after the update) ----
        if K == 1:
            dREX = XX[I]*c
            dIMX = -XX[I]*s
            print(f"I={I}, cos={c:.6f}, sin={s:.6f}, "
                  f"dREX={dREX:.6f}, dIMX={dIMX:.6f}, "
                  f"REX_before={REX[K]:.6f}, IMX_before={IMX[K]:.6f}")

        # BASIC lines 340, 350
        REX[K] += XX[I]*c
        IMX[K] += -XX[I]*s

        # 340
        # 350

    if K == 1:
        print(f" REX_after={REX[K]:.6f}, IMX_after={IMX[K]:.6f}")
# 370 NEXT I; 380 NEXT K

# Print the final values of REX and IMX
print("-----")
print("Final values of REX and IMX")
print("REX:", REX)
print("IMX:", IMX)
```

Printed trace output (program output for lines 310–380, $K = 1$)

```

I=0, cos=1.000000, sin=0.000000, dREX=1.000000, dIMX=-0.000000, REX_before=0.000000,
    REX_after=1.000000, IMX_after=0.000000
I=1, cos=0.500000, sin=0.866025, dREX=0.500000, dIMX=-0.866025, REX_before=1.000000,
    REX_after=1.500000, IMX_after=-0.866025
I=2, cos=-0.500000, sin=0.866025, dREX=-1.000000, dIMX=-1.732051, REX_before=1.500000,
    REX_after=0.500000, IMX_after=-2.598076
I=3, cos=-1.000000, sin=0.000000, dREX=-1.000000, dIMX=-0.000000, REX_before=0.500000,
    REX_after=-0.500000, IMX_after=-2.598076
I=4, cos=-0.500000, sin=-0.866025, dREX=-1.500000, dIMX=2.598076, REX_before=-0.500000,
    REX_after=-2.000000, IMX_after=-0.000000
I=5, cos=0.500000, sin=-0.866025, dREX=1.000000, dIMX=1.732051, REX_before=-2.000000,
    REX_after=-1.000000, IMX_after=1.732051

```

Final values of REX and IMX

REX: [10. -1. -2. 2. -2. -1.]

IMX: [0.00000000e+00 1.73205081e+00 -1.55431223e-15 -3.30778432e-15
-3.10862447e-15 -1.73205081e+00]

5 Question 5: Feature Extraction vs. Feature Matching

Feature extraction and feature matching are distinct, sequential stages in signal analysis. In essence, extraction creates a meaningful summary of the data, and matching uses that summary to identify the data.

Feature Extraction

This is the process of transforming raw, high-dimensional data (like an audio signal) into a concise and informative set of values called **features**. This step reduces the data's complexity while highlighting its most critical characteristics. A primary example is using the Discrete Fourier Transform (DFT) to convert a signal from the time domain to the frequency domain, where the amplitudes of the constituent sine and cosine waves become the signal's features.

Feature Matching

This process follows feature extraction. The extracted features from an unknown signal are compared to a pre-existing library or database of features from known signals. The system then searches for the closest match to classify or identify the unknown signal. For example, the frequency features of an unknown sound can be compared against the known frequency features of different vehicle engines to identify the vehicle type.

6 Question 6: Frequency Response of a System

A system's **frequency response** is formally defined as the Discrete Fourier Transform (DFT) of its impulse response. If a system's impulse response in the time domain is denoted by $h[n]$, its frequency response in the frequency domain is denoted by $H[f]$, where $H[f] = \text{DFT}\{h[n]\}$.

Conceptual Meaning

The frequency response provides a complete description of how a linear system affects sinusoidal signals passing through it. Specifically, it details how the **amplitude** and **phase** of each frequency component are modified by the system. It effectively answers the question: "Which frequencies does this system pass, block, or amplify?"

Purpose

While the impulse response characterizes a system in the time domain, it can often be difficult to interpret. The frequency response provides a much more intuitive understanding of the system's function, often revealing it to be a specific type of filter (e.g., low-pass, high-pass, or bandpass). For example, a system that blocks high frequencies will have a frequency response with low amplitudes at those high frequencies.

7 Question 7: The Fast Fourier Transform (FFT)

The FFT is an algorithm that provides a computationally efficient method for calculating the Discrete Fourier Transform (DFT).

Definition

The **Fast Fourier Transform (FFT)** is not a separate type of transform, but rather an optimized algorithm that drastically reduces the number of computations required to compute the DFT of a signal. It produces the exact same result as the direct DFT method, but does so much more quickly.

Application

The FFT should be used in any application where a DFT is required, especially when performance is critical. Its speed makes it indispensable for:

- **Real-time systems:** where signals must be processed with minimal delay, such as in live audio filtering or digital communications.
- **Large datasets:** where analyzing signals with many thousands or millions of points would be computationally prohibitive using a direct DFT.

Complexity

The computational complexity of the FFT is $O(N \log N)$, where N is the number of samples in the signal. This is a significant improvement over the direct DFT algorithm's complexity of $O(N^2)$, and it is the primary reason for the FFT's widespread use.

8 Question 8: Time Domain vs. Frequency Domain Analysis

The primary domain for analyzing a signal depends on whether its essential information is contained in the instantaneous values of its samples or in its underlying oscillatory patterns.

1. Number of pedestrians that walked through a door: Time Domain

Explanation: This is a counting process where the key information is the occurrence of discrete events in time. The analysis is concerned with when each event happens and the cumulative total, which are fundamentally time-based characteristics.

2. Detecting the word "log-off": Frequency Domain

Explanation: Speech recognition relies on identifying the unique spectral content of spoken words. The information that distinguishes "log-off" from other words is encoded in its specific pattern of frequencies, requiring analysis in the frequency domain.

3. Monitoring the presence/absence of a vehicle on a quiet country road: Time Domain

Explanation: This signal tracks a binary state over time. The critical information is when the state changes (a car arrives or leaves) and how long it remains in a particular state. This is a direct analysis of the signal's amplitude at each point in time.

4. Discriminating between a car and a bus: Frequency Domain

Explanation: A car and a bus produce distinct acoustic signatures based on their engines and mechanical properties. These differences are best characterized by their frequency spectra, including fundamental frequencies and harmonics. Distinguishing between them requires analyzing the signal in the frequency domain.

9 Question 9: Spectrogram

A spectrogram is a visual representation of a signal's frequency content as it evolves over time. It effectively provides a time-frequency analysis of the signal, making it ideal for non-stationary signals like audio.

Construction

The process of creating a spectrogram involves three main steps:

1. **Windowing:** The original signal is divided into a sequence of short, typically overlapping, time frames or windows.
2. **Fourier Transform:** The Fast Fourier Transform (FFT) is applied to each individual window to obtain its frequency spectrum. This spectrum reveals the amplitudes of the various frequencies present within that short time frame.

3. **Visualization:** The resulting spectra are aligned chronologically to form an image. The horizontal axis of the image represents time, the vertical axis represents frequency, and a third dimension, the amplitude of each frequency, is represented by color or intensity.

Interpretation

When viewing a spectrogram:

- The **x-axis** shows the progression of **time**.
- The **y-axis** shows the range of **frequencies**, typically from low to high.
- The **color or brightness** of a point indicates the **amplitude** of a specific frequency at a specific time. Often, darker or brighter areas represent strong frequency components, which in speech analysis are known as formants.

10 Question 10: Purpose of a Hamming Window

The main purpose of applying a **Hamming window** to a signal is to reduce an undesirable effect known as **spectral leakage** that occurs during the process of computing a Discrete Fourier Transform (DFT).

The Problem: Spectral Leakage

When a finite-length segment is taken from a signal for DFT analysis, the sharp truncation at the segment's beginning and end acts as an artificial discontinuity. These sharp transitions introduce distortion in the frequency domain, causing the energy of a specific frequency to spread out, or "leak," into adjacent frequency bins. This effect can mask weaker signals and reduce the ability to resolve closely spaced frequencies.

The Solution: Windowing

A Hamming window is a function that has a smooth, curved shape, tapering from a maximum value in the center down to small, non-zero values at the edges. By multiplying the signal segment by the Hamming window, the signal's amplitude is smoothly attenuated at its boundaries. This tapering minimizes the abrupt start-and-stop discontinuities, which significantly reduces spectral leakage. The resulting frequency spectrum is cleaner, more accurate, and has better resolution for identifying distinct frequency components.

11 Question 11: DFT Applications in a Smart City

The Discrete Fourier Transform (DFT) is essential for many smart city applications that rely on interpreting complex signals from the environment. Two such instances are:

1. Structural Health Monitoring of Bridges

Application: Accelerometers and strain gauges are placed on critical infrastructure like bridges to constantly record vibration data. The DFT is applied to this time-series vibration data to extract the bridge's resonant frequencies.

Benefit: Over time, a change or drift in these characteristic frequencies can serve as an early warning for structural damage, allowing municipal engineers to perform proactive maintenance and ensure public safety.

2. Acoustic Traffic and Noise Pollution Monitoring

Application: A network of microphones is deployed in a city to capture ambient sounds. The DFT is used to analyze the frequency spectrum of the recorded audio signals.

Benefit: This frequency data can be used to automatically classify different types of vehicles (cars, trucks, buses, emergency sirens) for advanced traffic flow analysis. Furthermore, it allows the city to monitor noise pollution levels across different frequency bands, helping to enforce noise ordinances and improve urban quality of life.

12 Question 12: Convolution and DFT in Autonomous Systems

Both convolution and the DFT are fundamental tools for processing signals in autonomous systems. The choice between them depends on whether the objective is to filter/transform the signal in the time domain (convolution) or to analyze its constituent frequencies (DFT).

1. Signal: Microphone Audio

- **Objective for Convolution: Filtering.** To reliably detect an emergency siren, the system must first isolate it from background noise. The objective is to use convolution to apply a digital band-pass filter to the raw audio, allowing only the frequencies typical of sirens to pass through.
- **Objective for DFT: Identification.** After filtering, the system must verify the sound's identity. The objective is to use the DFT to analyze the frequency spectrum of the filtered signal over time. This allows the system to identify the specific rising and falling frequency pattern that uniquely characterizes a siren.

2. Signal: LiDAR Data

- **Objective for Convolution: Edge Detection.** When LiDAR data is viewed as a 2D image (e.g., a top-down grid), the objective is to locate the edges of objects. A 2D convolution with an edge-detection kernel is used to process this image, highlighting the boundaries of other vehicles, pedestrians, and lane markings.
- **Objective for DFT: Road Quality Analysis.** Using a LiDAR sensor to measure the road profile directly beneath the vehicle generates a 1D signal of surface height versus time. The objective is to analyze this signal for an adaptive suspension system. The DFT is used to determine the frequency content of the road's roughness, enabling the system to adjust for optimal ride comfort.

13 Question 13: Convolution in Smart Health Applications

Convolution is widely used in smart health for processing sensor data and medical images to improve clarity and enable accurate automated analysis. Two key applications are:

1. Filtering Noise from Wearable ECG Monitors

Application: Wearable health sensors record electrocardiogram (ECG) signals to monitor heart activity. These raw signals are often corrupted by noise from patient movement and electrical interference.

Use of Convolution: To clean the signal, it is convolved with the impulse response of a digital filter (e.g., a band-pass filter). This operation selectively removes the noise while preserving the essential features of the cardiac signal. A cleaner signal allows for more reliable automated detection of heart rate, rhythm, and potential arrhythmias.

2. Edge Enhancement in Medical Imaging

Application: In fields like radiology, doctors and algorithms analyze medical images (such as MRI or CT scans) to detect pathologies like tumors or fractures. The clarity of edges and textures is critical for an accurate diagnosis.

Use of Convolution: A 2D convolution is applied to the image data. By convolving the image with a sharpening kernel (a small 2D matrix), the edges and fine details of anatomical structures are enhanced. This makes subtle abnormalities more apparent, improving the accuracy of both human and machine-based diagnostic systems.

References

- [1] *CS 6762: Signal Processing, Machine Learning and Control Class Materials and Lectures*. University of Virginia.
- [2] *Other Internet Sources for basic information, facts and examples*.

Acknowledgment

This Homework makes use of Generative AI models like Gemini and ChatGPT for research to form and refine answers as required for this homework and some of the main sources for the information are cited. The author acknowledges that they have not given or received unauthorized assistance on this homework. The Author also acknowledges that they have understood the course content and lectures for the preparation of these homework solutions.